

Lightweight API Security Monitoring and Anomaly Detection: Real-Time Detection of Five OWASP-Based Vulnerability Categories

Mrs. M.Shyni

Assistant Professor (SS)
Rajalakshmi Engineering College
Chennai, India
shyni.m@rajalakshmi.edu.in

Varsha B

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
prathikshathila@gmail.com

Tejushree Sanjeevi Kumar

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
tejushreebv@gmail.com

Valluru Varshini

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
valluruvarshini9@gmail.com

Abstract—API anomaly detection utilizing machine learning with Isolation Forest and SHAP has been shown to work successfully in an enterprise context. However, conventional methods usually score the traffic after the fact and explain each of the detected anomalies, which is not scalable when the number of alerts grows. This paper describes a lightweight solution that focuses on just five well defined attack types from OWASP API Security Top 10: broken authentication, unrestricted resource consumption, business flow abuse, broken object-level authorization (BOLA) and broken function-level authorization (BFLA). Each request is analyzed against these five types of attacks in real-time by using simple deterministic rules (Algorithm 1) instead of extracting machine learning inference on the detection path, and rule checking itself takes just 0.21 μ s on average, and total hot path time is 1.50 μ s. In the case study conducted with labelled data (80 normal requests, 60 combined-demo requests, and independent attacks in each class), we observed that four out of five classes obtained perfect marks in precision, recall, and F1 (1.00 in each case). The fifth class, business flow abuse, achieved a high recall score of 1.00 but low precision of 0.16 (F1 = 0.28). This low precision value was only encountered when past data was examined. Another observation was that we discovered authorization violations, for instance, BOLA, which were marked as statistically normal under Isolation Forest. This indicated that violations won't be exposed through traffic pattern anomaly detection.

I. INTRODUCTION

Modern cloud-based services are built around Application Programming Interfaces (APIs) which have become the target of cybercriminals because of their exposure on the public Internet. Earlier research has found out that some methods for unknown anomaly detection like Isolation Forest have been successfully used along with SHAP to detect strange API traffic in an enterprise environment [1]. Nevertheless, such systems usually work like offline solutions: the traffic is monitored, analyzed, and explained afterward and the detected anomalies are accounted for completely. This paper adopts a novel technique that is influenced by two practical concerns. Instead of depending upon a general anomaly score to determine what qualifies as “unusual,” detection is based on five specific attack categories from the OWASP API Security Top 10 white paper

[2]. One very widely used industry reference to the most popular ways that APIs get exploited is presented. Second, rather than scoring traffic in batches, each incoming request is identified using fast threshold tests, thus eliminating the requirement for active learning inference as part of the detection process. One machine learning model, Isolation Forest, is retained, but instead of being incorporated into the detection process, the model is used to power an on-demand explanation solution that is only invoked when an analyst decodes the flagged request. As a result, the ability to explain outcomes using SHAP is still preserved while excluding the computation of the explanations for every flagged event. The remainder of this paper is structured as follows: Section II provides a survey of works related to the topic; Section III describes the architecture of the system and outlines five target categories; Section IV discusses implementation issues; Section V presents an experimental study; Section VI compares the proposed method with other existing solutions; Section VII discusses limitations; Section VIII provides directions for future research.

II. RELATED WORK

Anomaly detection comprises various techniques used for monitoring the behavior of applications, including statistical methods, distance-based techniques, density-based methods, and data learning, all relying on different assumptions about the normal state of operation and the availability of labels [3]. The isolation forest technique is based on separating the comprehended subjects through a number of random partitions without assessing the distribution of normal data [4], [5]. For the purpose of comparison, it is also essential to mention such unsupervised techniques as local outlier factor (LOF) [6] and single-class support vector machine [7]. Advanced versions of the isolation forest algorithm make use of the hyperplanes that are randomly oriented to prevent potential issues with the axes alignment [8]. In the current research, the classic isolation forest method is employed; it should be noted that it is employed only

for the purpose of needing the explanation, rather than the task of detection. There is a similar application developed by Mittal et al. that uses the isolation forest method coupled with the shap library to detect threats in APIs and reports 0.94 precision and 0.91 recall achieved with the help of simulation of fintech companies traffic streams [1]. The presented model, like other models that use the anomaly detection technique. The OWASP API Security Top 10 [2] documents the most common API vulnerability classes observed in practice, including Broken Object Level Authorization, Broken Authentication, Broken Function Level Authorization, and Unrestricted Resource Consumption. Although widely referenced in industry security guidance, this taxonomy is less commonly used as the direct basis for a real-time, rule-based detection architecture in the anomaly-detection literature, which more often relies on general-purpose statistical scoring.

For local interpretability, LIME approximates a model's behavior around an individual prediction using an interpretable surrogate [10], whereas SHAP provides an additive feature-attribution framework [11]. TreeExplainer introduced efficient, tree-specific attribution algorithms [12]. Explanations require caution, as Slack et al. demonstrate that adversarially constructed classifiers can mislead perturbation-based LIME and SHAP explanations [13].

Operational intrusion detection also requires attention to analyst workload. Sommer and Paxson discuss why successful machine-learning experiments do not automatically translate into effective deployed intrusion detection [14]. The analyst-in-the-loop AI2 system combines outlier detection with analyst feedback and supervised learning [15], providing context for the dashboard and feedback loop discussed as future work in Section VIII.

III. PROPOSED METHODOLOGY

A. System Overview

The system proposed focuses on five categories chosen from the OWASP API Security Top 10 based on their applicability to the given type of endpoints in the simulated backend. The selected categories are: broken authentication, unregulated resource consumption, business flow misuse, broken object-level authorization (BOLA), and broken function-level authorization (BFLA). The incoming request is evaluated against all the categories in an inline fashion before the actual response is produced. Additionally, another Isolation Forest model is used only to provide an on-demand SHAP explanation

for a particular flagged request when necessary.

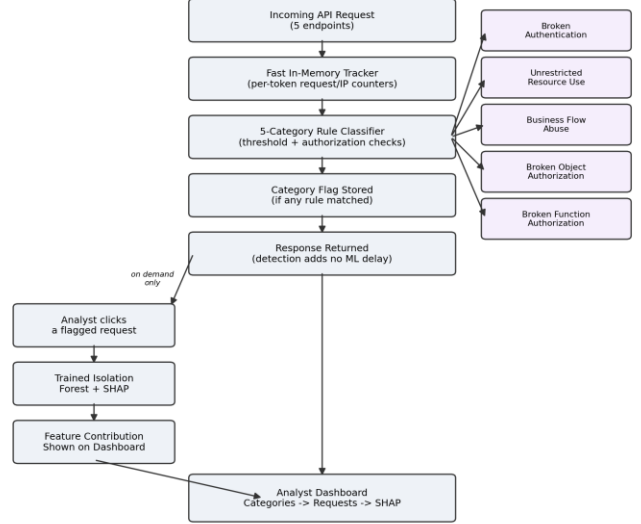


Fig. 1. Real-time category detection pipeline (center/right) with on-demand SHAP explanation (left).

B. Fast, In-Memory Behavioral Tracking

Two out of five categories heavily rely on recent behavior history such as the number of requests made by the token or the number of different IP addresses it has been associated with. To eliminate the need for constant recomputation of these figures by querying through request records considering increasing traffic rates, there are small rolling windows of short-term memory maintained per token that store a deque of timestamps and a set of IP addresses recently used. Thus, this allows making each verification within a constant time frame.

C. Category Detection Rules

The Twentieth Category: Detection Rules

- **Broken authentication:** This rule gets triggered when the same authentication token comes from three or more different IP addresses within a period of five minutes, which is an indication of token sharing, replays or even token theft.

- **Unrestricted resource consumption:** This rule gets activated when a token goes beyond a certain threshold of requests within a minute or when it goes over the allowed request size.

- **Business flow abuse:** This rule comes into play when certain action endpoint (checkout and profile_update) is invoked excessively when compared to other endpoints.

- **Broken object-level authorization:** This rule gets triggered when a request targets a resource identifier which does not belong to authenticated principal, which can be verified by means of the ownership check.

- **Broken function-level authorization:** This rule gets activated when a token without admin rights tries to call an admin endpoint.

In summary, the three former rules are behaviorbased, while two latter rely on the fact that requests permissions are compared against their real rights.

Algorithm 1: Per-request category classification

Input: endpoint e , payload size s , requests_last_60s r , distinct_ips_last_5min d , bola_flag b , bfla_flag f .
Output: set of matched categories (0, 1, or many).
1. $flags \leftarrow \{\}$
2. if $d \geq 3$: $flags \leftarrow flags \cup \{broken_authentication\}$
3. if $r \geq 10$: $flags \leftarrow flags \cup \{unrestricted_resource_use\}$
else if $s \geq 2000$: $flags \leftarrow flags \cup \{unrestricted_resource_use\}$
4. if $e \in \{checkout, profile_update\}$ and $r \geq 5$:
 $flags \leftarrow flags \cup \{business_flow_abuse\}$
5. if b : $flags \leftarrow flags \cup \{broken_object_authorization\}$
6. if f : $flags \leftarrow flags \cup \{broken_function_authorization\}$
7. return $flags$

D. On-Demand SHAP Explanation

For categories 1-3, the salient behavioral aspects of the requests, i.e., payload size, response time, request rate, and unique IP count, are suitable input for a trained Isolation Forest model. Rather than automatically generating a SHAP explanation for each flagged request, it is only generated when an analyst chooses a specific request from the dashboard using the decision function of the model and SHAP explainer specific to that request's feature vector. Thus, the need for computation is reduced while keeping the possibility of using explanations as a diagnostic tool. BOLA and BFLA violations, however, cannot be explained using this kind of approach: BOLA requests cannot be differentiated in most instances from normal requests based on the equivalent criteria of size and time of the request. It is violated because of the person submitting the request, and not because of what the request is.

E. Analyst Dashboard

The output will be demonstrated through a dashboard on the web with three levels. The first level will consist of one card for every category, showing the number of flagged applications and the latest instance as well. The second level will give all flagged requests under a certain category when pressing the category. Such requests will include all necessary data: endpoint, method, source IP, token ID, date & time, and the reason of the rule execution. The third level will show the SHAP justification of the flagged request if it is requested.

F. Computational Complexity

The in-memory tracker (described in Section III-B) is built such that its resource cost per request does not increase with the traffic volume, unlike the previous solution that recalculated token behavior by going through the request log for each check. Each timestamp added to a token deque is removed at most once when it goes outside the 60-second period, which means that the total number of operations W done on N requests is limited by $W(N) \leq N$ (1), resulting in the amortized request cost of $W(N)/N = O(1)$ (2). The distinct IP address table is operated similarly and has the amortized cost of $O(m)$, where m is the number of IP addresses associated with that token in the 5-minute window; m was at most 5 in all tests in Section V. Together with a constant number of operations shown in Algorithm 1, this leads to $O(1)$ amortized cost under the assumption that m remains small, which is exactly what is demonstrated in Table II (1.5 μ s).

IV. IMPLEMENTATION

The implementation of the entire system is done through Python with the use of FastAPI for the backend layer of API, SQLAlchemy and SQLite as the persistent storage, and through the use of scikit-learn packages to include the Isolation Forest model for instant explanations. The feature of tracking the behavior in real-time is done through thread-safe rolling windows for every token, using deques for keeping time stamps and dictionaries for IP addresses recently used that helps avoid repeated database lookups on the path for detection. The access tokens are created by the means of use of JSON Web Encryption method with the support of the algorithm AES-256-GCM and are of short expiry.

V. RESULTS

A. Detection Outcomes on Live Traffic

The evaluation was performed with the help of simulated traffic against a backend working in real-time: normal traffic was created to simulate direct interaction by human users and was followed by five intentionally generated cases for every targeted class. Separate credentials were used to prevent the sharing of statistical information across different cases. Thus, in total, there were 60 requests with their statuses obtained.

The summary of the results is given in Table I: all five classes were identified correctly, showing the reasons for the detection perfectly correlated with the rules applied. For those on the basis of BOLA, SHAP explanation was computed to find out whether the Isolation Forest algorithm would generate detection without being trained on the data previously. It turned out that the detection score is a normal one indicating that the request is normal considering payload dimension, timing, and frequency. Thus, the conclusion can be made that violations of the authorization nature cannot be detected through only traffic shapes. In contrast, the explanation of one of the operating cases, the oversized payload one, was negative showing that the payload dimension was the most possible reason confirming the efficiency of the explanation mechanism for the cases based on the categories

TABLE I
CATEGORY DETECTION RESULTS

Category	Count	Representative Reason
Broken authentication	3	Token used from 5 distinct IPs in 5 minutes
Unrestricted resource use	12	21 requests from one token in 60 seconds
Business flow abuse	12	8 calls to profile_update in 60 seconds
Broken object authorization	3	Requested a record not owned by the caller
Broken function authorization	1	Non-admin token called an admin-only endpoint

B. Classification Latency

In order to validate that classification is adequately quick for the detection route as asserted in Section III, the cost was

measured directly while preventing Algorithm 1 from using the in-memory tracker that comes before this algorithm in the live middleware. The results of the measurements conducted in 20,000 timed calls are displayed in Table II.

TABLE II
CLASSIFICATION LATENCY (μ S, $N = 20,000$)

Stage	Mean	Median	P95	P99
Rule evaluation only (Alg. 1)	0.21	0.17	0.32	0.38
Full hot path (tracker + Alg. 1)	1.50	1.36	2.15	3.41

Rule checks by themselves require approximately 0.21 microseconds, while the full hot path, inclusive of updating of the in-memory tracker (adding and eliminating of the deque of timestamps per token and the IP set) lasts about 1.50 microseconds. Both these values are negligible compared to the usual time required by network and database I/O for making requests, while no trained Isolation Forest model is needed for performing detection at request time, confirming that rule detection is not dependent on any machine-learning inference made at that time.

C. Precision, Recall, and F1

The numbers in Table I indicate that there were a total of five detected categories under traffic handling humans, but that these do not provide the number of false positives on their own. A separate analysis was conducted under ground truth conditions with 80 normal requests and five separate attack types, with one being conducted for each category using a different token for each. True positives, false positives, and false negatives were determined for each category as detailed below. The precision, recall, and F1 scores are calculated according to the following formulae:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

The results are given in Table III for each of the categories tested.

TABLE III
PRECISION, RECALL, AND F1 PER CATEGORY

Category	Prec.	Recall	F1
Broken authentication	1.00	1.00	1.00
Unrestricted resource use	1.00	1.00	1.00
Business flow abuse	0.16	1.00	0.28
Broken object authorization	1.00	1.00	1.00
Broken function authorization	1.00	1.00	1.00

Perfect recall and precision were obtained in four of the categories. Except Business flow abuse, which had a low precision (0.16), resulting from false positives only: as this evaluation involved the deliberate compression of time (with requests being sent back to back and without the intervals of several seconds that simulate human behavior in the demonstration described in Section V-A), a few ordinary tokens gathered enough requests over 60 seconds in the shared

counter and managed to cross the threshold for business flow abuse based upon a profile_update or checkout request. This situation could not happen in the real-life demonstration (Table I) because no false positives were recorded, as the real-life demonstration made it possible to keep the shared request-counting window low enough to avoid abuse threshold crossing. Thus, this finding is explicitly stated rather than put aside: it allows one to conclude that the threshold applied to business flow abuse is sensitive to elapsed time and supports the ideas of severity threshold adjustment and proper calibration outlined in Section VII.

D. Explaining the Business-Flow Precision Result

The 0.16 accuracy recorded in Section V-C can be calculated directly from the way in which the benchmark was created. There are ten normal tokens that make $K=8$ requests quickly, where there are 4 endpoint types and only 2 types contribute to the business-flow rule (checkout and profile_update), meaning the per-request likelihood of getting to the endpoint is $p=2/4=0.5$. Since all K requests happen in the same 60-second interval, the shared request counter reaches the limits $\theta=5$ at the 5th request; therefore, every other business-flow-endpoint call from that token, requests 5-8 will be flagged. The expected number of these calls for each token is basically p times the number of requests which can be served. Thus, when $T = 10$ tokens are taken into account the expected number of false positive becomes $E[\text{FP}] = T \cdot p \cdot \max(0, K - \theta + 1)$ (6). When one puts $T = 10$, $p = 0.5$, $K = 8$, $\theta = 5$ into this formula we receive $E[\text{FP}] = 10 \cdot 0.5 \cdot 4 = 20$ which is very close to our practical observation of 21 (Table III). Given the fact that $\text{TP} = 4$ is constant due to the scenario the precision we can expect is $\text{TP} / (\text{TP} + E[\text{FP}]) = 4 / 24 \approx 0.17$ (7). This final value is just one negative result different than the actual value of 0.16 (4/25). This means that it can be said for sure that lower precision is not the defect of implementation but the result of analytics.

E. Throughput Bound Under Load

Since the hot path specified in Table II ($c = 1.50 \mu$ s per request) occurs synchronously before the response is returned, continuous request rates approaching its inverse lead to a queue delay on top of c . Assuming the detection process is modeled as a single service station with deterministic service time c and arrival rate R , the volume of traffic can be represented as

$$\rho = R \cdot c \quad (8)$$

and according to Pollaczek-Khinchine formula, the mean waiting time for M/D/1 queue can be calculated as

$$W_q = \rho / (2(1 - \rho)) \cdot c, \text{ for } \rho < 1 \quad (9)$$

thus the expected total time of detection is

$$c + W_q = c \cdot (1 + \rho / (2(1 - \rho))) \quad (10)$$

In order for this to stay within 10% of c only ($\rho / (2(1 - \rho)) \leq 0.1$), we have to have $\rho \leq 0.167$, or $R \leq 0.167/c \approx 111,000$ requests per second for the investigated single core — many times greater than any expected API loads for the simulated backend, pointing to the fact that the detection process will unlikely be the bottleneck for the system.

VI. COMPARISON WITH EXISTING SYSTEMS

The differences are summarized in Table IV. The system does not claim superior performances over existing Isolation Forest-SHAP systems regarding anomaly detection. Instead, the goal of the current system is to focus on a set of different pre-defined classes of critical vulnerabilities with fast, deterministic checks and using machine learning to provide reasons for the occurrence and not to detect anomalies themselves.

TABLE IV
COMPARISON WITH EXISTING SYSTEM

Aspect	Existing System	Proposed System
Detection basis	General anomaly score	Five named OWASP categories
Detection timing	Periodic batch scoring	Real-time, inline per request
Explanation scope	Every flagged anomaly	On-demand, per analyst request
Authorization violations	Not addressed	Explicit rule-based checks
Detection-path ML inference	Required	Not required (rule-based)

VII. LIMITATIONS

Certain limitations have to be recognized. First, instead of using real-world production traffic, evaluation was done using simulated traffic; attackers may not behave the same way as envisioned based on the values used in the study. Next, detection is restricted only to the five chosen categories. An attack falling outside this range, for example, an injection attack, would not be considered, as there is no general-purpose anomaly score on the detection route. Third, the category thresholds, such as request-rate limits, are viewed as constants; these can lead to a decrease in the precision when assessing business flow abuse based on the results of the evaluation done under compressed time, as illustrated in Section V-C; while no false positives were found in real life.

ACKNOWLEDGMENT

Fig. 1 in this paper was generated using Claude (Anthropic), an AI tool. All other content, including the implementation, experiments, and results, is the authors' own work.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we introduced a novel API threat detection system that ensures real-time detection and is based on OWASP API Security Top 10 principles. The proposed system encompasses a combination of conventional detection methodologies based on API-specific behavioral and authorization practices with the use of SHAP-based explanations for cases when analysts want to investigate any specific incident. We conducted experiments to prove that our detection methodology worked correctly and instantly detected malicious activities in all five categories. The experiments confirmed that our system successfully detect

anomalies in traffic flow, however, in terms of detection of authorization issues like BOLA, we emphasize that traffic anomaly techniques alone cannot detect breaches thus leading us to hybrid approach that we suggested in this paper. Future work will be focused on calibration of adaptive thresholds, development of analyst feedback system for marking incidents as resolved, provision of real-time reports, involvement of token behavior in detection process, provision of token ciphering system as separate topic and testing of our system in real setups.

REFERENCES

- [1] A. Mittal, P. Keshap, and A. Hosabettu, "AI-driven real-time API security: Explainable threat detection for cloud environments," in *Proc. 2025 IEEE Int. Carnahan Conf. Security Technology (ICCST)*, Oct. 2025, pp. 1–7, doi: 10.1109/ICCST63435.2025.11293949.
- [2] OWASP Foundation, "OWASP API Security Top 10," 2023. [Online]. Available: <https://owasp.org/API-Security/>
- [3] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, Dec. 2008, pp. 413–422.
- [5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, 2012.
- [6] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, May 2000, pp. 93–104.
- [7] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [8] S. Hariri, M. C. Kind, and R. J. Brunner, "Extended isolation forest," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1479–1489, 2021.
- [9] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.09089*, 2018.
- [10] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Aug. 2016, pp. 1135–1144.
- [11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [12] S. M. Lundberg *et al.*, "Explainable AI for trees: From local explanations to global understanding," *arXiv preprint arXiv:1905.04610*, 2019.
- [13] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods," in *Proc. AAAI/ACM Conf. AI, Ethics, and Society*, Feb. 2020, pp. 180–186.
- [14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. 2010 IEEE Symp. Security and Privacy*, May 2010, pp. 305–316.
- [15] K. Veeramachaneni, I. Araldo, V. Korrapati, C. Bassias, and K. Li, "AP2: Training a big data machine to defend," in *Proc. 2016 IEEE 2nd Int. Conf. Big Data Security on Cloud (BigDataSecurity)*, Apr. 2016, pp. 49–54.